06. 상태 전이 (State Processing) 구현 분석

개요

Lighthouse의 상태 전이(State Processing) 시스템은 Ethereum 2.0 합의 프로토콜의 핵심 구성 요소로, 비콘 체인의 상태를 슬롯별, 블록별, 에포크별로 업데이트하는 역할을 담당합니다. `consensus/state_processing` 모듈은 이러한 상태 전이 로직을 체계적으로 구현하고 있으며, 각 처리 단계별로 명확하게 분리된 구조를 가지고 있습니다.

상태 전이는 다음 세 가지 주요 처리 단계로 구성됩니다:

1. **슬롯 처리 (Per-Slot Processing)**: 매 슬롯마다 실행되는 기본적인 상태 업데이트
2. **블록 처리 (Per-Block Processing)**: 새로운 블록이 도착했을 때 실행되는 블록 검증 및 상태 업데이트
3. **에포크 처리 (Per-Epoch Processing)**: 에포크 경계에서 실행되는 검증자 보상/처벌, 정당화/최종화 등의 처리

모듈 구조 및 아키텍처

전체 모듈 구조

```rust
// consensus/state_processing/src/lib.rs

pub mod per_slot_processing; // 슬롯별 처리

pub mod per_block_processing; // 블록별 처리

pub mod per_epoch_processing; // 에포크별 처리

pub mod state_advance; // 상태 전진 (빈 슬롯 처리)

pub mod upgrade; // 포크 업그레이드 처리
```

```
pub mod genesis; // 제네시스 상태 초기화
pub mod consensus_context; // 합의 컨텍스트 관리
```

주요 공개 함수들

```
// 핵심 상태 전이 함수들
pub use per_slot_processing::{per_slot_processing, Error as SlotProcessingError
pub use per_block_processing::{per_block_processing, BlockProcessingError};
pub use per_epoch_processing::{process_epoch as per_epoch_processing, EpochProc
```

슬롯 처리 (Per-Slot Processing)

핵심 함수: `per_slot_processing`

슬롯 처리는 매 슬롯마다 실행되는 가장 기본적인 상태 전이 과정입니다.

```
// consensus/state_processing/src/per_slot_processing.rs
pub fn per_slot_processing<E: EthSpec>(
state: &mut BeaconState<E>,
state_root: Option<Hash256>,
spec: &ChainSpec,
) -> Result<Option<EpochProcessingSummary<E>>, Error>
```

주요 처리 과정

1. **포크 검증**: 현재 상태가 올바른 포크 버전인지 확인
2. **상태 캐싱**: 이전 슬롯의 상태 루트와 블록 루트를 캐시에 저장
3. **에포크 경계 처리**: 새로운 에포크 시작 시 에포크 처리 실행
4. **슬롯 증가**: 상태의 슬롯 번호를 1 증가
5. **포크 업그레이드**: 포크 에포크에 도달한 경우 상태 업그레이드 실행

상태 캐싱 로직

```
fn cache_state<E: EthSpec>(
state: &mut BeaconState<E>,
state_root: Option<Hash256>,
) -> Result<(), Error> {
// 이전 상태 루트 계산 또는 사용
let previous_state_root = if let Some(root) = state_root {
root
} else {
state.update_tree_hash_cache()? // 트리 해시 캐시 업데이트
};

// 슬롯을 임시로 증가시켜 getter/setter 함수 사용
let previous_slot = state.slot();
state.slot_mut().safe_add_assign(1)?;

// 이전 슬롯의 상태 루트 저장
```

```
state.set_state_root(previous_slot, previous_state_root)?;


// 최신 블록 헤더의 상태 루트 업데이트
if state.latest_block_header().state_root == Hash256::zero() {
state.latest_block_header_mut().state_root = previous_state_root;
}


// 블록 루트 캐시
let latest_block_root = state.latest_block_header().canonical_root();
state.set_block_root(previous_slot, latest_block_root)?;


// 슬롯을 원래 값으로 복원
state.slot_mut().safe_sub_assign(1)?;
Ok(())
}
```

포크 업그레이드 처리

```
// 에포크 경계에서 포크 업그레이드 확인 및 실행
if state.slot().safe_rem(E::slots_per_epoch())? == 0 {
// Altair 포크 업그레이드
if spec.altair_fork_epoch == Some(state.current_epoch()) {
upgrade_to_altair(state, spec)?;
}
```

```
// Bellatrix 포크 업그레이드
if spec.bellatrix_fork_epoch == Some(state.current_epoch()) {
upgrade_to_bellatrix(state, spec)?;
}
// Capella 포크 업그레이드
if spec.capella_fork_epoch == Some(state.current_epoch()) {
upgrade_to_capella(state, spec)?;
}
// 추가 포크들...
}
```

블록 처리 (Per-Block Processing)

핵심 함수: `per_block_processing`

블록 처리는 새로운 블록을 받았을 때 해당 블록을 검증하고 상태에 적용하는 과정입니다.

```
// consensus/state_processing/src/per_block_processing.rs
pub fn per_block_processing<E: EthSpec, Payload: AbstractExecPayload<E>>(
state: &mut BeaconState<E>,
signed_block: &SignedBeaconBlock<E, Payload>,
block_signature_strategy: BlockSignatureStrategy,
verify_block_root: VerifyBlockRoot,
ctxt: &mut ConsensusContext<E>,
```

```
spec: &ChainSpec,
) -> Result<(), BlockProcessingError>
```

블록 처리 순서

1. **포크 검증**: 블록과 상태의 포크 버전 일치 확인
2. **캐시 초기화**: 에포크 캐시, 진행 균형 캐시, 슬래싱 캐시 구축
3. **서명 검증**: 블록 서명 전략에 따른 서명 검증
4. **블록 헤더 처리**: 블록 헤더 검증 및 상태 업데이트
5. **실행 페이로드 처리**: 실행 레이어 페이로드 처리 (Merge 이후)
6. **RANDAO 처리**: RANDAO 공개값 검증 및 믹스 업데이트
7. **Eth1 데이터 처리**: Eth1 데이터 투표 처리
8. **오퍼레이션 처리**: 블록 내 모든 오퍼레이션 처리
9. **동기화 집계 처리**: 동기화 위원회 집계 처리 (Altair 이후)

블록 헤더 처리

```
pub fn process_block_header<E: EthSpec>(
state: &mut BeaconState<E>,
block_header: BeaconBlockHeader,
verify_block_root: VerifyBlockRoot,
ctxt: &mut ConsensusContext<E>,
spec: &ChainSpec,
) -> Result<u64, BlockOperationError<HeaderInvalid>> {
// 슬롯 일치 검증
```

```
verify!(
block_header.slot == state.slot(),
HeaderInvalid::StateSlotMismatch
);

// 블록이 최신 블록보다 새로운지 검증
verify!(
block_header.slot > state.latest_block_header().slot,
HeaderInvalid::OlderThanLatestBlockHeader
);

// 제안자 인덱스 검증
let proposer_index = block_header.proposer_index;
let state_proposer_index = ctxt.get_proposer_index(state, spec)?;
verify!(
proposer_index == state_proposer_index,
HeaderInvalid::ProposerIndexMismatch
);

// 부모 블록 루트 검증
if verify_block_root == VerifyBlockRoot::True {
let expected_previous_block_root = state.latest_block_header().tree_hash_root()
verify!(
```

```
block_header.parent_root == expected_previous_block_root,
HeaderInvalid::ParentBlockRootMismatch
);
}

// 제안자가 슬래시되지 않았는지 검증
verify!(
!state.get_validator(proposer_index  as usize)?.slashed,
HeaderInvalid::ProposerSlashed(proposer_index)
);

// 상태 업데이트
state.slashings_cache_mut().update_latest_block_slot(block_header.slot);
*state.latest_block_header_mut() = block_header;

Ok(proposer_index)
}
```

오퍼레이션 처리

블록 내의 다양한 오퍼레이션들을 순차적으로 처리합니다.

```
// consensus/state_processing/src/per_block_processing/process_operations.rs
pub  fn  process_operations<E: EthSpec, Payload: AbstractExecPayload<E>>(
```

```
state: &mut BeaconState<E>,
block_body: BeaconBlockBodyRef<E, Payload>,
verify_signatures: VerifySignatures,
ctxt: &mut ConsensusContext<E>,
spec: &ChainSpec,
) -> Result<(), BlockProcessingError> {
// 제안자 슬래싱 처리
process_proposer_slashings(state, block_body.proposer_slashings(), verify_signat
// 증명자 슬래싱 처리
process_attester_slashings(state, block_body.attester_slashings(), verify_signat
// 증명 처리
process_attestations(state, block_body, verify_signatures, ctxt, spec)?;
// 예치금 처리
process_deposits(state, block_body.deposits(), spec)?;
// 자발적 탈퇴 처리
process_exits(state, block_body.voluntary_exits(), verify_signatures, spec)?;

// BLS to Execution 변경 처리 (Capella 이후)
if  let Ok(bls_to_execution_changes) = block_body.bls_to_execution_changes() {
process_bls_to_execution_changes(state, bls_to_execution_changes, verify_signatu
}

// 실행 요청 처리 (Electra 이후)
```

```
if state.fork_name_unchecked().electra_enabled() {
state.update_pubkey_cache()?;
let execution_requests = block_body.execution_requests()?;
process_deposit_requests(state, &execution_requests.deposits, spec)?;
process_withdrawal_requests(state, &execution_requests.withdrawals, spec)?;
process_consolidation_requests(state, &execution_requests.consolidations, spec)?
}

Ok(())
}
```

증명 처리 (Attestation Processing)

증명 처리는 포크에 따라 다른 방식으로 처리됩니다:

Base 포크 (Phase 0) 처리:

```
// Base 포크에서는 PendingAttestation으로 저장
pub fn process_attestations<'a, E: EthSpec, I>(
state: &mut BeaconState<E>,
attestations: I,
verify_signatures: VerifySignatures,
ctxt: &mut ConsensusContext<E>,
spec: &ChainSpec,
) -> Result<(), BlockProcessingError> {
```

```
let  proposer_index = ctxt.get_proposer_index(state, spec)?;


for (i, attestation) in  attestations.enumerate() {
verify_attestation_for_block_inclusion(state, attestation, ctxt, verify_signatur


let  pending_attestation = PendingAttestation {
aggregation_bits: attestation.aggregation_bits.clone(),
data: attestation.data.clone(),
inclusion_delay: state.slot().safe_sub(attestation.data.slot)?.as_u64(),
proposer_index,
};


// 현재 또는 이전 에포크 증명으로 분류하여 저장
if  attestation.data.target.epoch == state.current_epoch() {
state.as_base_mut()?.current_epoch_attestations.push(pending_attestation)?;
} else {
state.as_base_mut()?.previous_epoch_attestations.push(pending_attestation)?;
}
}
Ok(())
}
```

Altair 이후 포크 처리:

```
// Altair 이후에는 참여 플래그로 즉시 처리
pub fn process_attestation<E: EthSpec>(
state: &mut BeaconState<E>,
attestation: AttestationRef<E>,
att_index: usize,
ctxt: &mut ConsensusContext<E>,
verify_signatures: VerifySignatures,
spec: &ChainSpec,
) -> Result<(), BlockProcessingError> {
let indexed_att = verify_attestation_for_block_inclusion(state, attestation, ct

// 참여 플래그 인덱스 계산
let data = attestation.data();
let inclusion_delay = state.slot().safe_sub(data.slot)?.as_u64();
let participation_flag_indices = get_attestation_participation_flag_indices(sta

// 각 증명자에 대해 참여 플래그 업데이트
let mut proposer_reward_numerator = 0;
for index in indexed_att.attesting_indices_iter() {
let index = *index as usize;
for (flag_index, &weight) in PARTICIPATION_FLAG_WEIGHTS.iter().enumerate() {
if participation_flag_indices.contains(&flag_index) {
```

```
let epoch_participation = state.get_epoch_participation_mut(data.target.epoch,
let validator_participation = epoch_participation.get_mut(index)?;
if !validator_participation.has_flag(flag_index)? {
validator_participation.add_flag(flag_index)?;
proposer_reward_numerator.safe_add_assign(state.get_base_reward(index)?.safe_mul
}
}
}
}

// 제안자 보상 계산 및 지급
let proposer_reward = proposer_reward_numerator.safe_div(proposer_reward_denomi
increase_balance(state, proposer_index as usize, proposer_reward)?;
Ok(())
}
```

에포크 처리 (Per-Epoch Processing)

핵심 함수: `process_epoch`

에포크 처리는 에포크 경계에서 실행되는 복잡한 상태 전이 과정입니다.

```
// consensus/state_processing/src/per_epoch_processing.rs
pub fn process_epoch<E: EthSpec>(
```

```
state: &mut BeaconState<E>,
spec: &ChainSpec,
) -> Result<EpochProcessingSummary<E>, Error> {
// 포크에 따라 다른 처리 방식 선택
if state.fork_name_unchecked().altair_enabled() {
altair::process_epoch(state, spec)
} else {
base::process_epoch(state, spec)
}
}
```

Base 포크 에포크 처리

```
// consensus/state_processing/src/per_epoch_processing/base.rs
pub fn process_epoch<E: EthSpec>(
state: &mut BeaconState<E>,
spec: &ChainSpec,
) -> Result<EpochProcessingSummary<E>, Error> {
// 위원회 캐시 구축
state.build_committee_cache(RelativeEpoch::Previous, spec)?;
state.build_committee_cache(RelativeEpoch::Current, spec)?;
state.build_committee_cache(RelativeEpoch::Next, spec)?;
state.build_total_active_balance_cache(spec)?;
```

```
initialize_epoch_cache(state, spec)?;


// 검증자 상태 분석
let mut validator_statuses = ValidatorStatuses::new(state, spec)?;
validator_statuses.process_attestations(state)?;


// 정당화 및 최종화 처리
let justification_and_finalization_state =
process_justification_and_finalization(state, &validator_statuses.total_balances
justification_and_finalization_state.apply_changes_to_state(state);


// 보상 및 처벌 처리
process_rewards_and_penalties(state, &validator_statuses, spec)?;


// 레지스트리 업데이트 (활성화/탈퇴)
process_registry_updates(state, spec)?;


// 슬래싱 처리
process_slashings(state, validator_statuses.total_balances.current_epoch(), spec


// 각종 리셋 처리
process_eth1_data_reset(state)?; // Eth1 데이터 투표 리셋
process_effective_balance_updates(state, spec)?; // 유효 잔액 업데이트
```

```
process_slashings_reset(state)?; // 슬래싱 리셋
process_randao_mixes_reset(state)?; // RANDAO 믹스 리셋
process_historical_roots_update(state)?; // 히스토리컬 루트 업데이트
process_participation_record_updates(state)?; // 참여 기록 업데이트

// 캐시 회전
state.advance_caches()?;

Ok(EpochProcessingSummary::Base {
total_balances: validator_statuses.total_balances,
statuses: validator_statuses.statuses,
})
}
```

Altair 포크 에포크 처리

```
// consensus/state_processing/src/per_epoch_processing/altair.rs
pub fn process_epoch<E: EthSpec>(
state: &mut BeaconState<E>,
spec: &ChainSpec,
) -> Result<EpochProcessingSummary<E>, Error> {
// 캐시 구축
state.build_committee_cache(RelativeEpoch::Previous, spec)?;
```

```
state.build_committee_cache(RelativeEpoch::Current, spec)?;
state.build_committee_cache(RelativeEpoch::Next, spec)?;
state.build_total_active_balance_cache(spec)?;
initialize_epoch_cache(state, spec)?;
initialize_progressive_balances_cache::<E>(state, spec)?;


let  sync_committee = state.current_sync_committee()?.clone();


// 정당화 및 최종화
let  justification_and_finalization_state = process_justification_and_finalizati
justification_and_finalization_state.apply_changes_to_state(state);


// 단일 패스로 여러 처리를 동시에 수행 (성능 최적화)
// - 비활성 업데이트
// - 보상 및 처벌
// - 레지스트리 업데이트
// - 슬래싱
// - 유효 잔액 업데이트
let  participation_summary = process_epoch_single_pass(state, spec, SinglePassCo


// 각종 리셋 및 업데이트
process_eth1_data_reset(state)?;
process_slashings_reset(state)?;
```

```
process_randao_mixes_reset(state)?;
// 히스토리컬 요약 또는 루트 업데이트 (Capella 전후 구분)
if  state.historical_summaries().is_ok() {
process_historical_summaries_update(state)?; // Post-Capella
} else {
process_historical_roots_update(state)?; // Pre-Capella
}

// 참여 플래그 업데이트
process_participation_flag_updates(state)?;
// 동기화 위원회 업데이트
process_sync_committee_updates(state, spec)?;

// 캐시 회전 및 진행 균형 업데이트
state.advance_caches()?;
update_progressive_balances_on_epoch_transition(state, spec)?;

Ok(EpochProcessingSummary::Altair {
progressive_balances: current_epoch_progressive_balances,
current_epoch_total_active_balance,
participation: participation_summary.into(),
sync_committee,
})
```

```
}
```

상태 전진 (State Advance)

빈 슬롯 처리

블록이 없는 슬롯들을 건너뛰어 상태를 특정 슬롯까지 전진시키는 기능입니다.

```
// consensus/state_processing/src/state_advance.rs


/// 완전한 상태 전진 - 모든 중간 상태 루트를 계산
pub fn complete_state_advance<E: EthSpec>(
state: &mut BeaconState<E>,
mut state_root_opt: Option<Hash256>,
target_slot: Slot,
spec: &ChainSpec,
) -> Result<(), Error> {
check_target_slot(state.slot(), target_slot)?;


while state.slot() < target_slot {
let state_root_opt = state_root_opt.take();
per_slot_processing(state, state_root_opt, spec).map_err(Error::PerSlotProcessin
}
Ok(())
```

```
}

/// 부분적 상태 전진 - 중간 상태 루트를 계산하지 않음 (성능 최적화)
pub fn partial_state_advance<E: EthSpec>(
state: &mut BeaconState<E>,
state_root_opt: Option<Hash256>,
target_slot: Slot,
spec: &ChainSpec,
) -> Result<(), Error> {
check_target_slot(state.slot(), target_slot)?;

let mut initial_state_root = Some(if state.slot() > state.latest_block_header
state_root_opt.unwrap_or_else(Hash256::zero)
} else {
state_root_opt.ok_or(Error::StateRootNotProvided)?
});

while state.slot() < target_slot {
let state_root = initial_state_root.take().unwrap_or_else(Hash256::zero);
per_slot_processing(state, Some(state_root), spec).map_err(Error::PerSlotProcess
}
Ok(())
```

```
}
```

포크 업그레이드 처리

포크 전환 메커니즘

각 포크 업그레이드는 에포크 경계에서 실행되며, 상태 구조를 새로운 포크 버전으로 변환합니다.

```rust
// consensus/state_processing/src/upgrade.rs
pub use altair::upgrade_to_altair;
pub use bellatrix::upgrade_to_bellatrix;
pub use capella::upgrade_to_capella;
pub use deneb::upgrade_to_deneb;
pub use electra::upgrade_to_electra;
pub use fulu::upgrade_to_fulu;
```

Altair 업그레이드 예시

```rust
// consensus/state_processing/src/upgrade/altair.rs
pub fn upgrade_to_altair<E: EthSpec>(
pre_state: &mut BeaconState<E>,
spec: &ChainSpec,
) -> Result<(), Error> {
let epoch = pre_state.current_epoch();
// 새로운 Altair 필드들 초기화
```

```
let  previous_epoch_participation = translate_participation(pre_state, epoch.saf

let  current_epoch_participation = translate_participation(pre_state, epoch, spe

// 동기화 위원회 초기화

let  current_sync_committee = get_next_sync_committee(pre_state, spec)?;

let  next_sync_committee = get_next_sync_committee(pre_state, spec)?;



// 비활성 점수 초기화

let  inactivity_scores = VariableList::new(vec![0u64; pre_state.validators().len



// 상태를 Altair 버전으로 변환

*pre_state = pre_state.upgrade_to_altair(

current_epoch_participation,

previous_epoch_participation,

inactivity_scores,

current_sync_committee,

next_sync_committee,

spec,

)?;



Ok(())

}
```

성능 최적화 기법

1. 단일 패스 처리 (Single Pass Processing)

Altair 이후 포크에서는 여러 에포크 처리 단계를 하나의 루프에서 처리하여 성능을 최적화합니다.

```
// consensus/state_processing/src/per_epoch_processing/single_pass.rs
pub fn process_epoch_single_pass<E: EthSpec>(
state: &mut BeaconState<E>,
spec: &ChainSpec,
config: SinglePassConfig,
) -> Result<ParticipationEpochSummary, Error> {
// 모든 검증자를 한 번의 루프로 처리
for (index, validator) in state.validators().iter().enumerate() {
// 비활성 점수 업데이트
process_inactivity_update(state, index, &participation_cache, spec)?;
// 보상 및 처벌 계산
process_rewards_and_penalties_for_validator(state, index, &participation_cache,
// 레지스트리 업데이트 (활성화/탈퇴)
process_registry_update_for_validator(state, index, spec)?;
// 슬래싱 처리
process_slashing_for_validator(state, index, total_balance, spec)?;
// 유효 잔액 업데이트
process_effective_balance_update_for_validator(state, index, spec)?;
```

```
}

Ok(participation_summary)

}
```

2. 진행 균형 캐시 (Progressive Balances Cache)

검증자 잔액 변화를 효율적으로 추적하기 위한 캐시 시스템입니다.

```
// 진행 균형 캐시 업데이트

pub fn update_progressive_balances_on_attestation<E: EthSpec>(

state: &mut BeaconState<E>,

target_epoch: Epoch,

flag_index: usize,

validator_effective_balance: u64,

validator_slashed: bool,

) -> Result<(), BeaconStateError> {

if let Ok(progressive_balances) = state.progressive_balances_cache_mut() {

progressive_balances.on_new_attestation(

target_epoch,

flag_index,

validator_effective_balance,

validator_slashed,

)?;

}
```

```
Ok(())
}
```

3. 병렬 처리

일부 검증 작업은 병렬로 처리하여 성능을 향상시킵니다.

```
// 예치금 머클 증명을 병렬로 검증
deposits.par_iter().enumerate().try_for_each(|(i, deposit)| {
verify_deposit_merkle_proof(
state,
deposit,
state.eth1_deposit_index().safe_add(i as u64)?,
spec,
).map_err(|e| e.into_with_index(i))
})?;
```

에러 처리 및 검증

계층적 에러 구조

```
#[derive(Debug, PartialEq)]
pub enum Error {
BeaconStateError(BeaconStateError),
EpochProcessingError(EpochProcessingError),
```

```
ArithError(ArithError),
InconsistentStateFork(InconsistentFork),
}
```

검증 매크로

```
// 블록 검증을 위한 매크로
macro_rules!  block_verify {
($condition:expr, $error:expr) => {
if !$condition {
return Err($error);
}
};
}

// 사용 예시
block_verify!(
block_header.slot == state.slot(),
BlockProcessingError::HeaderInvalid(HeaderInvalid::StateSlotMismatch)
);
```

실제 사용 예시

블록 처리 전체 흐름

```
// beacon_node에서 새 블록 처리
pub fn process_new_block<E: EthSpec>(
state: &mut BeaconState<E>,
signed_block: &SignedBeaconBlock<E>,
spec: &ChainSpec,
) -> Result<(), BlockProcessingError> {
let mut ctxt = ConsensusContext::new(state.slot());
// 블록 처리 실행
per_block_processing(
state,
signed_block,
BlockSignatureStrategy::VerifyBulk, // 모든 서명을 일괄 검증
VerifyBlockRoot::True, // 블록 루트 검증
&mut ctxt,
spec,
)?;
Ok(())
}
```

상태 전진 사용

```
// 빈 슬롯들을 건너뛰어 특정 슬롯까지 상태 전진

pub fn advance_state_to_slot<E: EthSpec>(

state: &mut BeaconState<E>,

target_slot: Slot,

state_root: Option<Hash256>,

spec: &ChainSpec,

) -> Result<(), Error> {

if state.slot() < target_slot {

complete_state_advance(state, state_root, target_slot, spec)?;

}

Ok(())

}
```

결론

Lighthouse의 상태 전이 시스템은 Ethereum 2.0 합의 프로토콜의 복잡한 요구사항을 체계적이고 효율적으로 구현하고 있습니다. 슬롯별, 블록별, 에포크별 처리를 명확히 분리하고, 각 포크별 특성을 고려한 구현을 통해 프로토콜의 진화에 유연하게 대응할 수 있는 구조를 제공합니다.

특히 Altair 이후 도입된 단일 패스 처리, 진행 균형 캐시, 병렬 처리 등의 최적화 기법들은 대규모 검증자 네트워크에서도 효율적인 상태 전이를 가능하게 합니다. 이러한 구현은 Lighthouse가 프로덕션 환경에서 안정적이고 성능이 우수한 합의 클라이언트로 동작할 수 있는 기반을 제공합니다.